Voting cards



- Take one at the beginning of the lecture.
- Please, **do not crumple them nor write on them**.
- Please, **return them** at the end of the lecture.

How is the group project going?

I don't have a group yet.

I don't know where to start.

Not everything is clear, but things are starting.

Everything is smooth.

Authorization

- **Definition**: Verifying the rights of access to a resource.

Summary of the lecture

- Access Control
- OAuth
- Attacks on Authorizations
- Design Princples

Access Control

Access Control

- **Definition**: The selective process of controlling access to the resources of a system.

Access Control

- **Definition**: The selective process of controlling access to the resources of a system.
- *Often composed of:*

Access Control

- **Definition**: The selective process of controlling access to the resources of a system.
- *Often composed of:*
  - *Authentication (& Identity Management)*

Access Control

- **Definition**: The selective process of controlling access to the resources of a system.
- *Often composed of:*
  - *Authentication (& Identity Management)*
  - ***Authorization***

Access Control

- **Definition**: The selective process of controlling access to the resources of a system.
- *Often composed of:*
  - *Authentication (& Identity Management)*
  - ***Authorization***
  - *Audit*

Access Control Matrices

Access Control Matrix: Example

	Patient 1 record	**Patient 2 record**	**Schedule**	**Prescription**
Doctor 1	Read, Append		Read, Edit	Read, Edit, Forward
Doctor 2		Read, Append	Read, Edit	Read, Edit, Forward
Receptionist			Read, Edit	Forward
Patient 1	Read			

Access Control Matrix: Example

Subjects:
Entities that can perform actions.

	Patient 1 record	**Patient 2 record**	**Schedule**	**Prescription**
Doctor 1	Read, Append		Read, Edit	Read, Edit, Forward
Doctor 2		Read, Append	Read, Edit	Read, Edit, Forward
Receptionist			Read, Edit	Forward
Patient 1	Read			

Access Control Matrix: Example

Objects:
Entities on which actions can be performed.

	Patient 1 record	Patient 2 record	Schedule	Prescription
Doctor 1	Read, Append		Read, Edit	Read, Edit, Forward
Doctor 2		Read, Append	Read, Edit	Read, Edit, Forward
Receptionist			Read, Edit	Forward
Patient 1	Read			

Access Control Matrix: Example

Rights:
Set of actions that can be performed.

Here: { Read, Append, Edit, Forward }

	Patient 1 record	Patient 2 record	Schedule	Prescription
Doctor 1	Read, Append		Read, Edit	Read, Edit, Forward
Doctor 2		Read, Append	Read, Edit	Read, Edit, Forward
Receptionist			Read, Edit	Forward
Patient 1	Read			

Access Control Matrix: Example

Access Control List (ACL):
For a given object, lists subjects' rights.

	Patient 1 record	**Patient 2 record**	**Schedule**	**Prescription**
Doctor 1	Read, Append		Read, Edit	Read, Edit, Forward
Doctor 2		Read, Append	Read, Edit	Read, Edit, Forward
Receptionist			Read, Edit	Forward
Patient 1	Read			

Access Control Matrix: Example

Capabilities:
For a given subjects, lists rights per object.

	Patient 1 record	Patient 2 record	Schedule	Prescription
Doctor 1	Read, Append		Read, Edit	Read, Edit, Forward
Doctor 2		Read, Append	Read, Edit	Read, Edit, Forward
Receptionist			Read, Edit	Forward
Patient 1	Read			

Do you think an entity can be both a subject and an object in the same system?

Yes

No

Different flavors of Access Control

Access Control List vs Capability-Based Access Control

Access Control List

- ACL is an approach to Access Control, where the system keeps track for each object, the list of subjects who has rights over said object.
- When a subject attempts to perform an operation, the system checks that the subject is in the list and indeed has the right to perform the operation over this object.

Capability-Based Access Control

- CBAC is an approach to Access Control, where the system focuses on checking if a subject has the capacity to perform an operation.
- Typically implemented with tokens. For a given right on an object a token is generated. If a subject has the token, they are allowed to perform the operation.
- Example: physical keys, google drive edit links.

Which of these approach is mainly implemented for the file system's permissions of Linux?

Access Control List

Capability-Based Access Control

Discretionary Access Control (DAC) vs Mandatory Access Control (MAC)

Discretionary Access Control (DAC)

- DAC is an approach to Access control where users of the system can set rights of some resources (Typically the one they own.).
- Example: Linux file system.
- Capability based systems can be described as DAC, as a capability can typically be transfered.

Mandatory Access Control (MAC)

- MAC is an approach to Access control where users of the system **cannot** set rights of resources on their own. Policies decide the rights at the scale of the system. (An administrator may be able to edit the rights.)
- Example: The University's room access control.

Role-Based Access Control and Attribute-Based Access Control

Role-Based Access Control (RBAC)

- If a system has a large number of users, managing rights at a very granular scale can be tedious.
- RBAC is an approach where roles are defined and given rights. Users are then given a role and inherit rights from their role.

Attribute-Based Access Control (ABAC)

- ABAC is similar to Role-Based approaches but allows more granular policies.
- A user can belong to different groups. (Or have different attributes)
- Rights can be set by declaring rules.

Key concepts (AC)

- Access Control
- Subjects / Objects / Rights
- ACL vs Capability Based Access Control
- DAC vs MAC (Who sets the rights)
- RBAC and ABAC (How to regroup subjects)

Linux File System Permissions

```
user@computer:~/shared$ ls -l
-rw-rw-r-- 1 user group 1138 Nov 12 11:13 my_file
-rwxrw-r-- 1 user group 1268 Nov 18 12:36 my_script.sh
-rwxrw-r-- 1 user group 1268 Nov 10 07:59 my_bin
drwxrwxr-x 2 user group 4096 Nov 18 09:48 mydir
```

```
user@computer:~/shared$ ls -l
-rw-rw-r-- 1 user group 1138 Nov 12 11:13 my_file
-rwxrw-r-- 1 user group 1268 Nov 18 12:36 my_script.sh
-rwxrw-r-- 1 user group 1268 Nov 10 07:59 my_bin
drwxrwxr-x 2 user group 4096 Nov 18 09:48 mydir
```

There are 3 different rights in Linux:
Read (r), Write (w), and eXecute (x)

```
user@computer:~/shared$ ls -l
-rw-rw-r-- 1 user group 1138 Nov 12 11:13 my_file
-rwxrw-r-- 1 user group 1268 Nov 18 12:36 my_script.sh
-rwxrw-r-- 1 user group 1268 Nov 10 07:59 my_bin
drwxrwxr-x 2 user group 4096 Nov 18 09:48 mydir
```

There are 3 different sets of Subjects:
The user (u) or owner of the file,
The group (g) to which the file belong,
And all (a), everyone else.

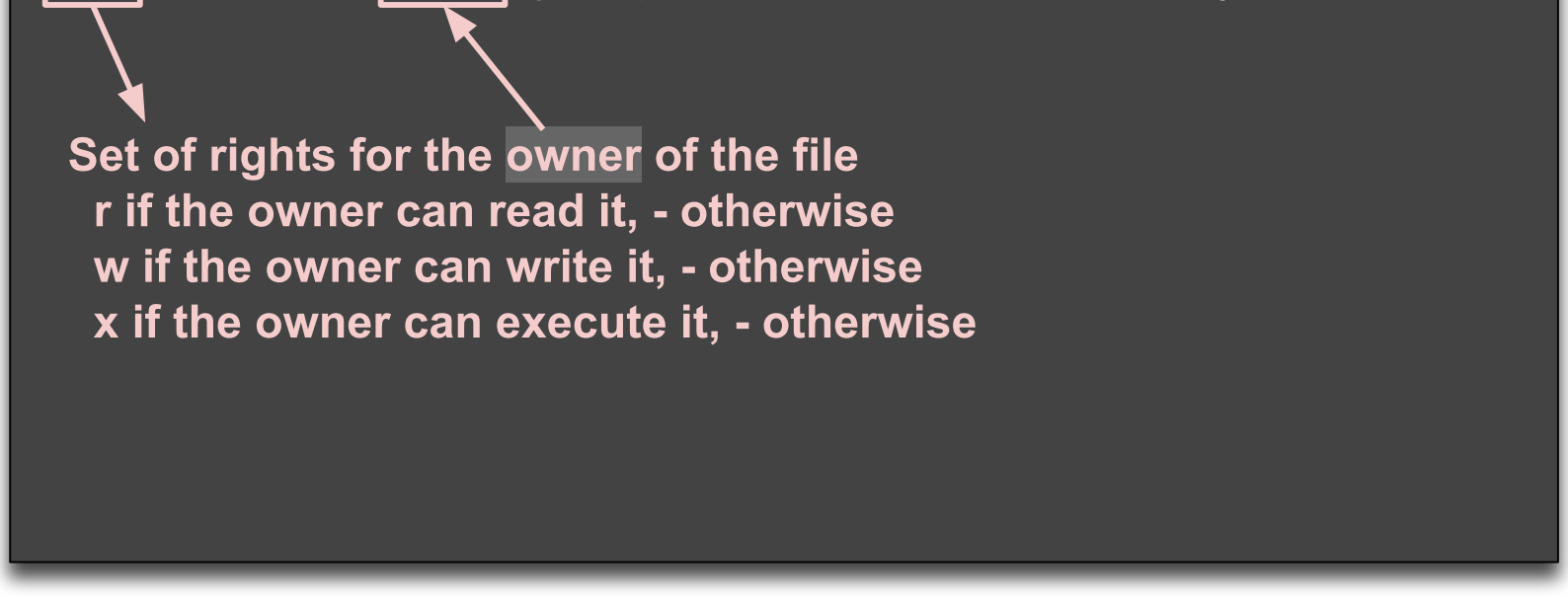
```
user@computer:~/shared$ ls -l
-rw-rw-r-- 1 user group 1138 Nov 12 11:13 my_file
```

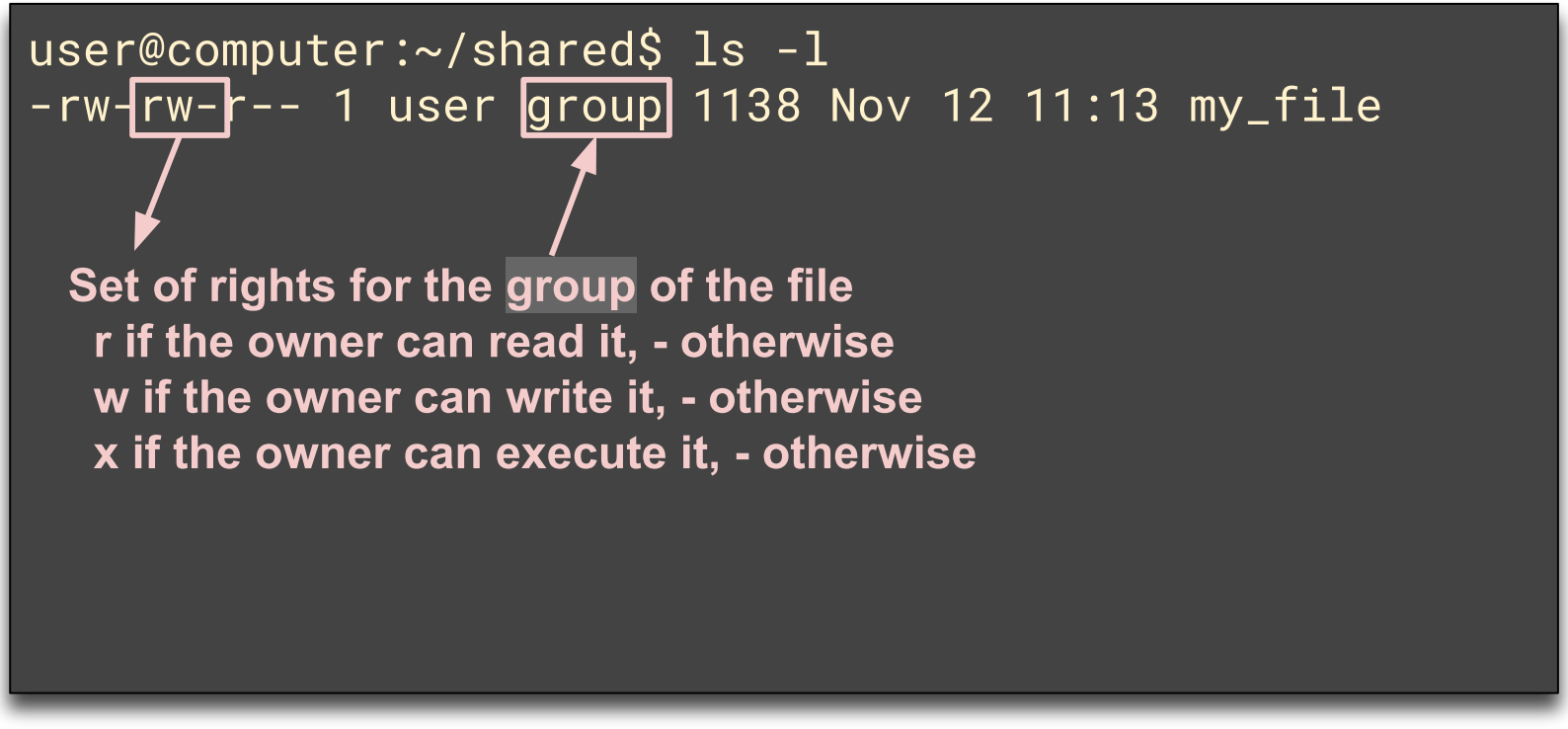
is this a directory?

d -> Yes

- -> No, it's a simple file.

```
user@computer:~/shared$ ls -l
-rw-rw-r-- 1 user group 1138 Nov 12 11:13 my_file
```




user@computer:~/shared$ ls -l
-rw-rw-r-- 1 user group 1138 Nov 12 11:13 my_file
Set of rights for the group of the file
r if the owner can read it, - otherwise
w if the owner can write it, - otherwise
x if the owner can execute it, - otherwise

```
user@computer:~/shared$ ls -l
-rw-rw-r-- 1 user group 1138 Nov 12 11:13 my_file
```



Set of rights for everyone else
- **r if the owner can read it, - otherwise**
- **w if the owner can write it, - otherwise**
- **x if the owner can execute it, - otherwise**

Since the owner of a file can change the permissions of their files, is it useful to set a file's permissions starting with ---? (No rights for the owners)

No, the owner won't be able to do anything.

No, the owner can change the permissions anyway.

Yes, even if the owner can change the permissions.

Linux permissions

- A process inherits the rights of the user who started it. (Unless specially configured otherwise)
- Access to resources are mostly managed through files (Hardware, etc..)
- For more fine-grained permissions, ACLs can be used in linux (See getfacl and setfacl).

Challenge 4.1 (Simple):

- I wrote a game for the student of the course. I put it on a Linux server where all students have an account and are part of the group students. Here are the files related to the game:
- - _ _ _ _ _ _ _ _ _ _ nicolas students game-binary
- - _ _ _ _ _ _ _ _ _ _ nicolas students high-scores.txt
- Set the rights to let anyone in the group students play the game and save their best score, without allowing anyone to cheat. (Apply the principle of least privilege and Fail-safe defaults). You can assume that a process regularly saves and flushes the content of high-scores.txt. (Credit for this problem: Alan Davidson.)

Android's Many Permissions

- Android controls what applications can do with a permission system.
- As opposed to Linux that manages what process can do through files and (rwx permissions), Android counts more than 300 permissions, ranging from hardware access to notifications rights.

OAuth

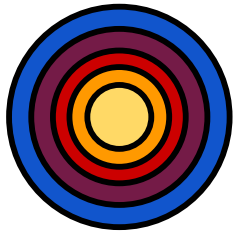


- OAuth is protocol to (among other things) delegate access to resources.
- RFC 6749 (IETF Standard)

OAuth flow: Example



User



A web application that generates beautiful visuals based on your music history.



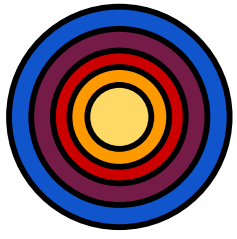
Music Service

A music on demand service, where you can buy music, listen to music, get access to your history, etc...

OAuth flow: Example



User
Resource Owner



Web App
Client

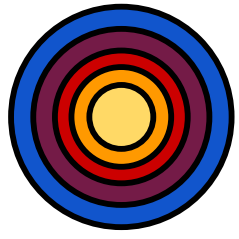


Music Service
Resource Server /
Authorization Server

OAuth flow: Example



User
Resource Owner



ClientID
Secret

Web App
Client



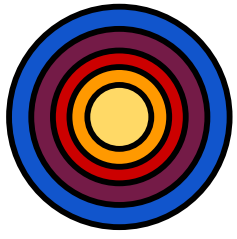
Music Service
Resource Server / Authorization Server

The web application needs to have a pre existing relationship with the Music Service. (The music service has given the web application a Client ID and a secret.)

OAuth flow: Example



User
Resource Owner

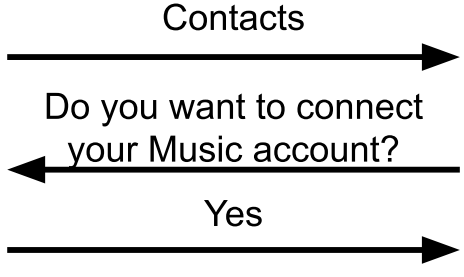


ClientID
Secret

Web App
Client



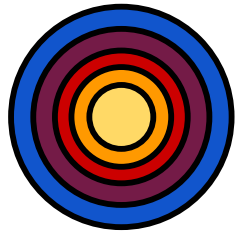
Music Service
Resource Server / Authorization Server



OAuth flow: Example



User
Resource Owner



ClientID
Secret

Web App
Client



Music Service
Resource Server / Authorization Server

Redirect to Music Service
(ClientID, callback URI, Scope)



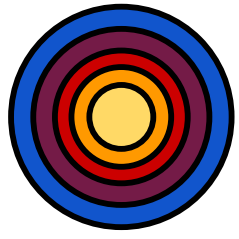
Can you give access to Web App to my music history ?
(ClientID, callback URI, Scope)

The **callback URI** is simply the address where the user can contact the Web App later with all the information necessary. The **scope** is the list of rights that the Web Application wants from the Music Service.

OAuth flow: Example



User
Resource Owner



ClientID
Secret

Web App
Client



Music Service
Resource Server / Authorization Server



Can you give access to Web App to my music history ?
(ClientID, callback URI, Scope)

First, login if you haven't yet.



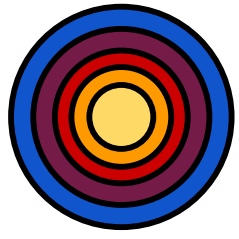


Login

OAuth flow: Example



User
Resource Owner



ClientID
Secret

Web App
Client



Music Service
Resource Server / Authorization Server



Can you give access to Web App to my music history ?
(ClientID, callback URI, Scope)



Consent: Do you want to give access to Web App to your history?

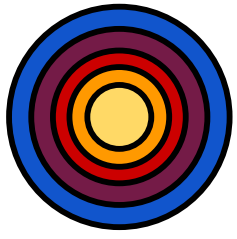


Yes

OAuth flow: Example



User
Resource Owner



ClientID
Secret

Web App
Client



Music Service
Resource Server / Authorization Server



Can you give access to Web App to my music history ?
(ClientID, callback URI, Scope)

Temporary Code



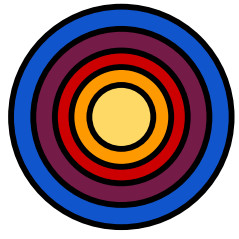
Temporary Code



OAuth flow: Example



User
Resource Owner



ClientID
Secret

Web App
Client



Music Service
Resource Server /
Authorization Server

Temporary Code

Temporary Code

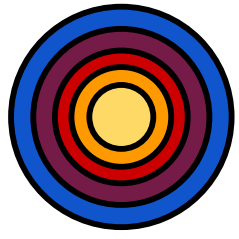
Can you give me an Authorization Token?
(Temporary Code, ClientID, Secret)

Token

OAuth flow: Example



User
Resource Owner



ClientID
Secret
Token

Web App
Client



Music Service
Resource Server / Authorization Server

Can you give me the history of **User** ?
(Token)



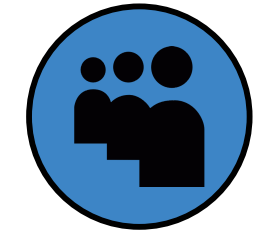
History



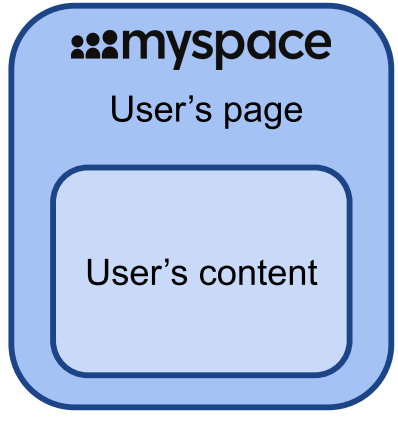
As long, as User doesn't **revoke** the token (by contacting Music Service), the Web Application can **access the resource** History, and **only** that resource.

Attacks on Authorization

How Samy (Kamkar) became many people's hero.



MySpace was (and still is) one of the first social networks with global outreach in the early 2000s.

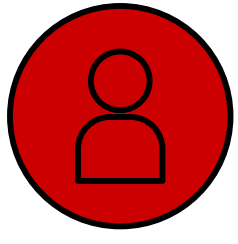


Among other things, it allowed users to customise their public page with their own content and formatting.

Samy is my Hero worm

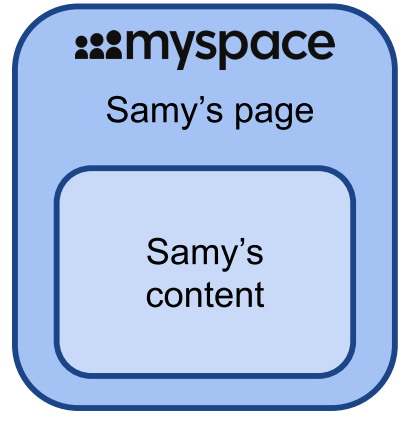


Samy found a way to bypass MySpace's input sanitation and put a small JavaScript program on his page.

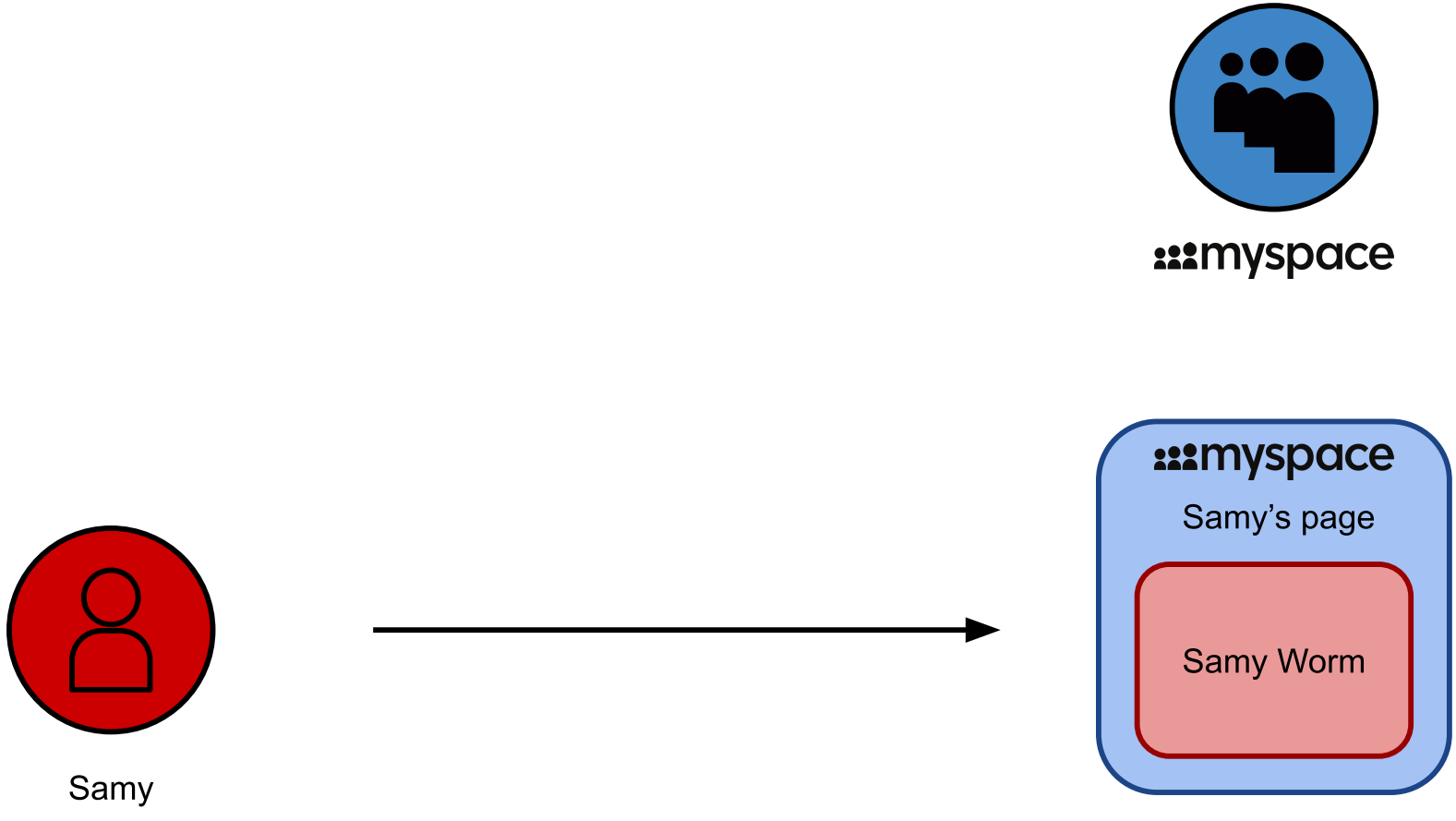


Samy

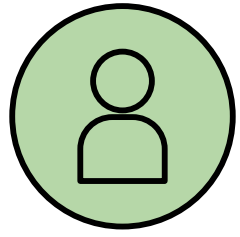




Samy is my Hero worm



Samy is my Hero worm

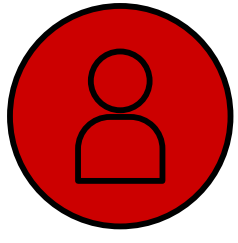


User

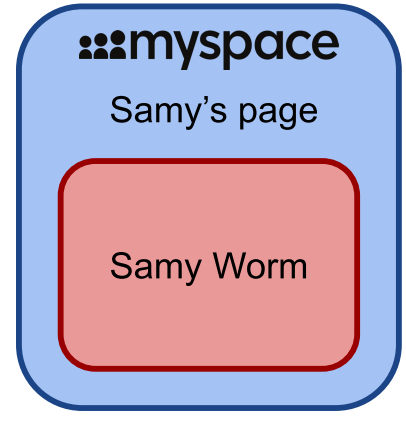
When users visit a web page,a small html & javascript program is downloaded and run by their browser.



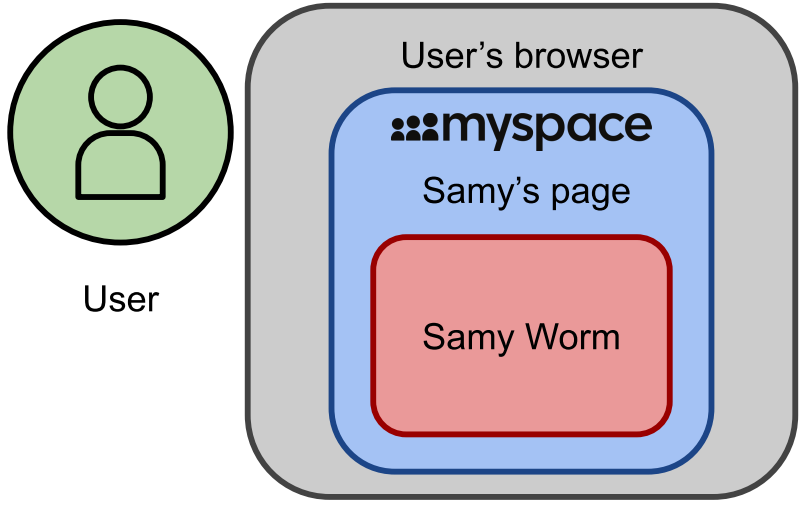
myspace



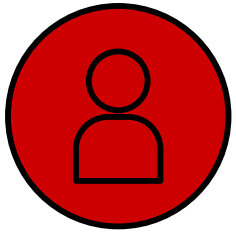
Samy



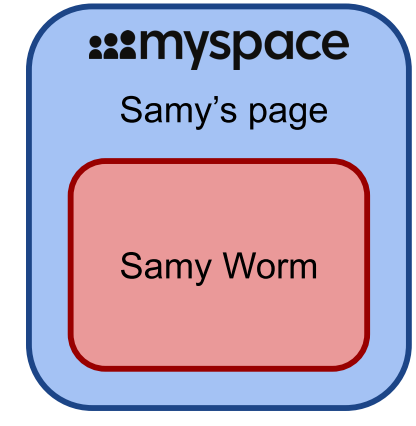
Samy is my Hero worm



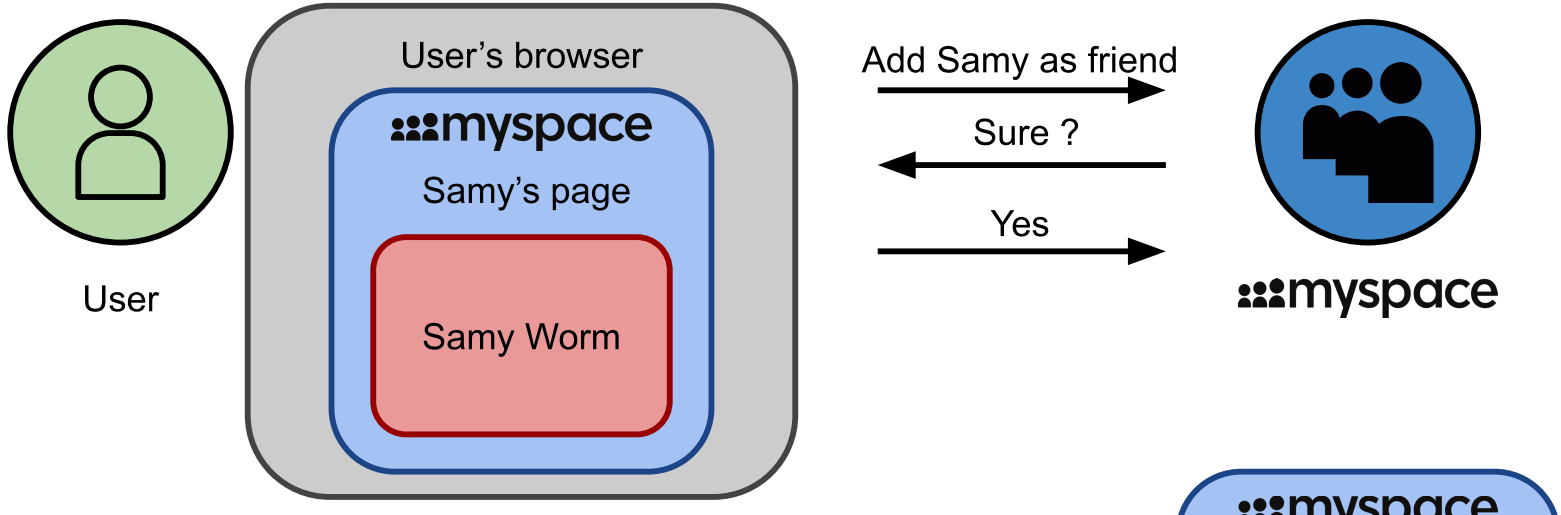




In this case, when a user (connected to myspace with their account) visited Samy’s page, they would run his piece of malicious code.



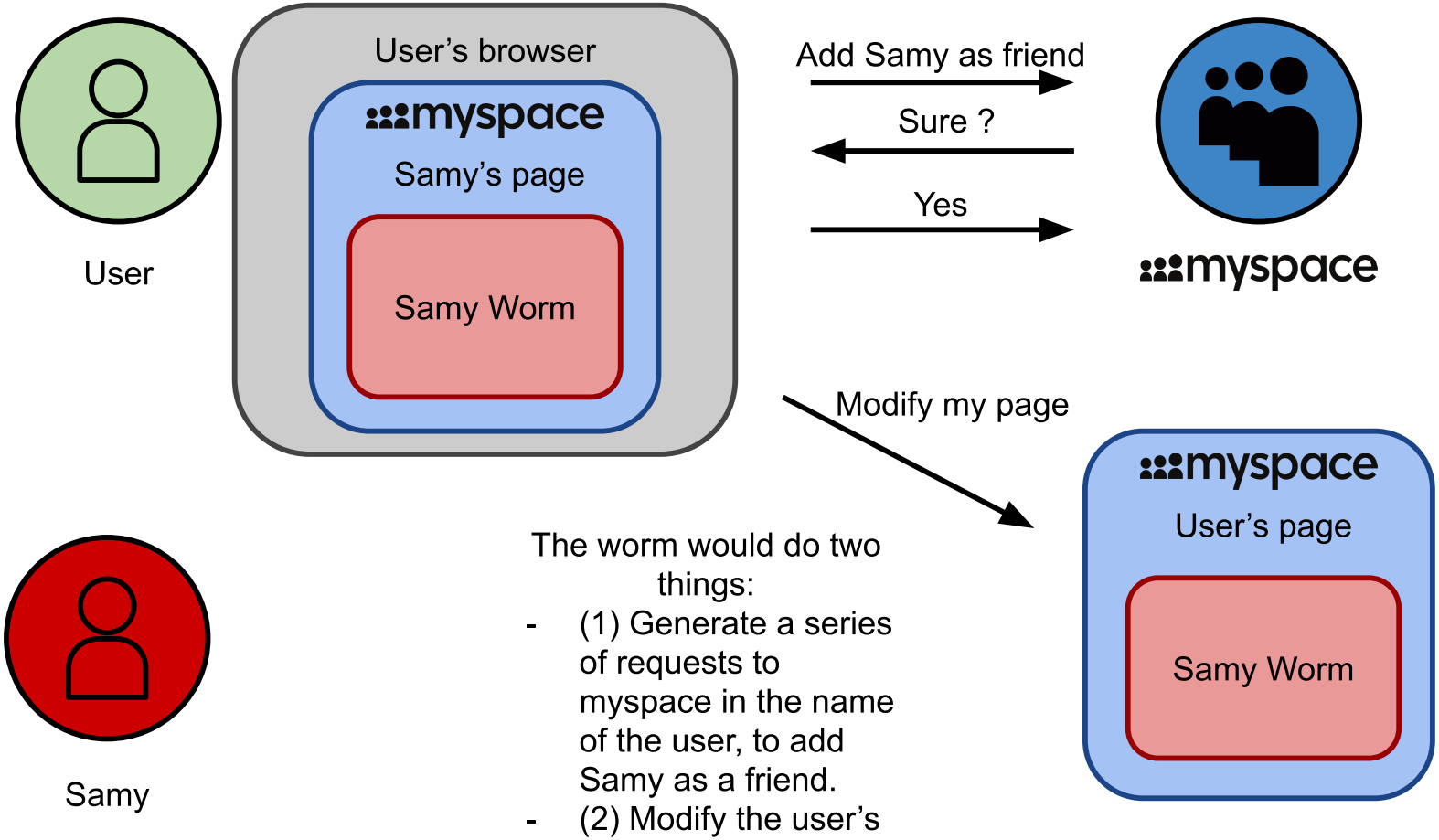
Samy is my Hero worm



The worm would do two things:

- (1) Generate a series of requests to myspace in the name of the user, to add Samy as a friend.

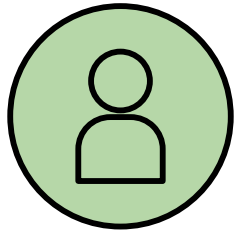
Samy is my Hero worm



The worm would do two things:

- (1) Generate a series of requests to myspace in the name of the user, to add Samy as a friend.
- (2) Modify the user's page to add the worm's code in the page. (And add "Samy is my hero in the profile)

Samy is my Hero worm

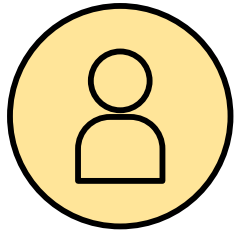


User



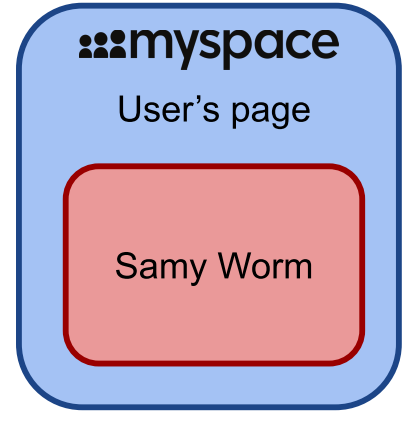
myspace

Any user that would now visit the infected's user's page, would be infected as well.

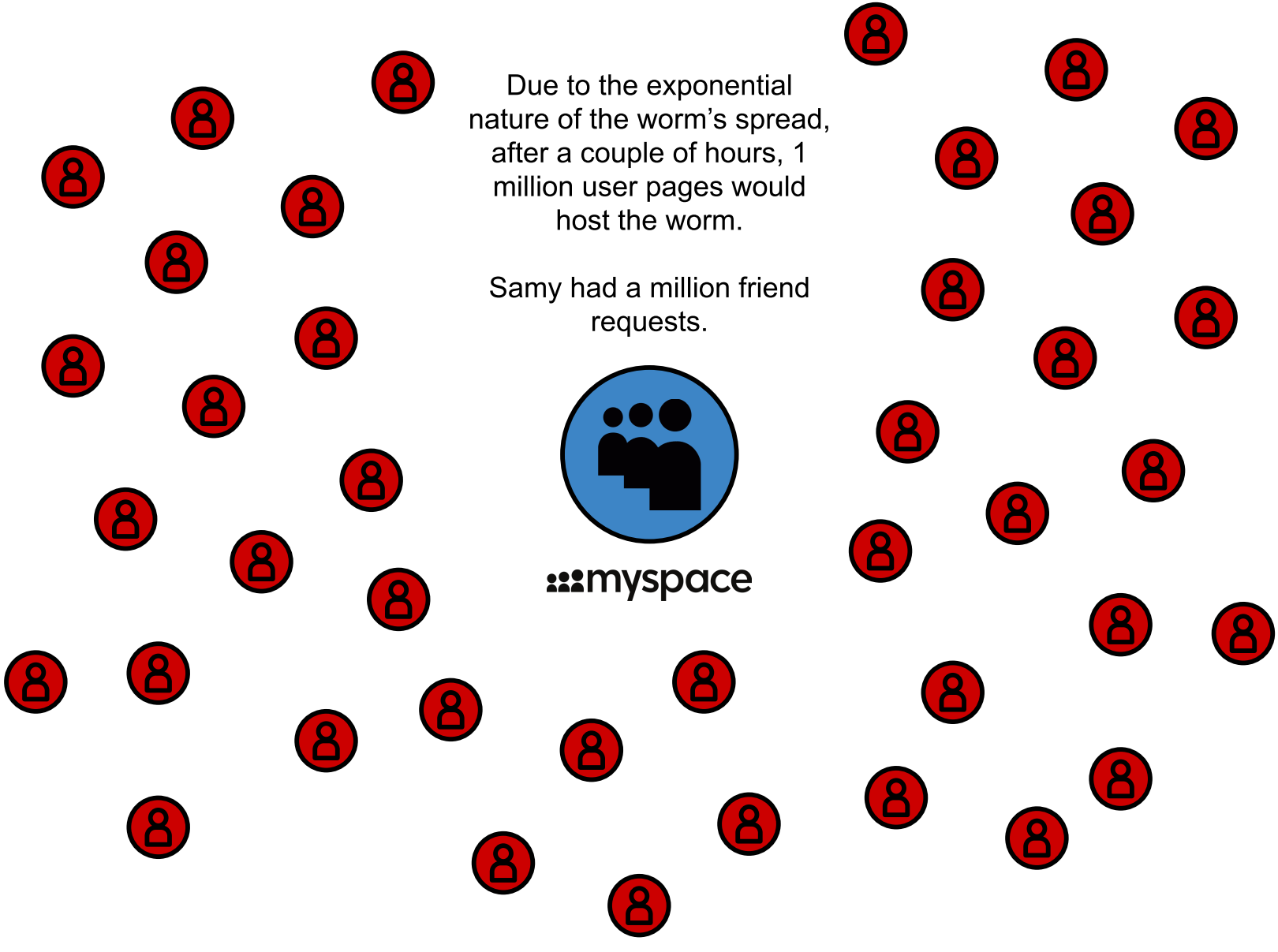


Any other user





Samy is my Hero worm

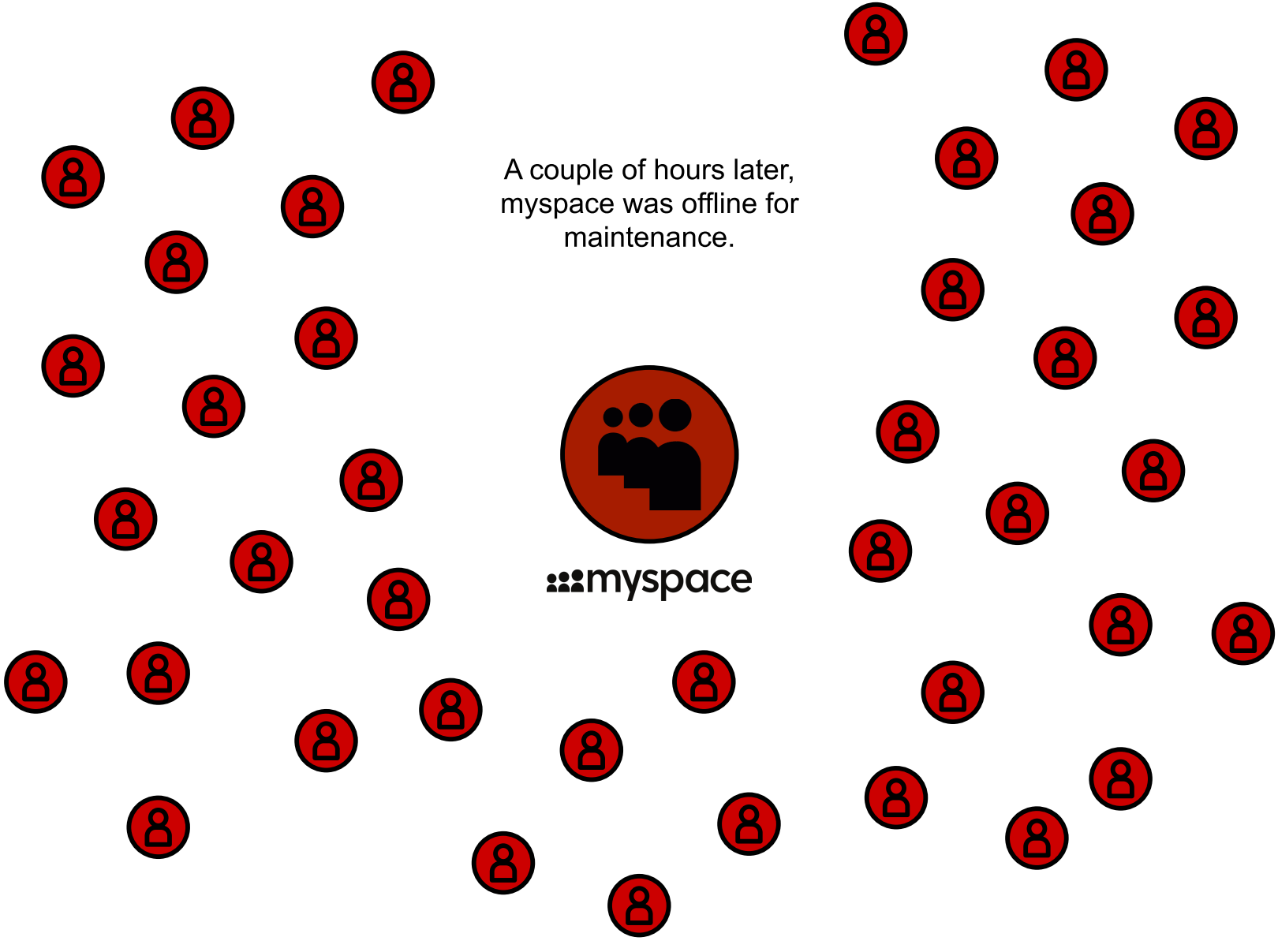


Due to the exponential nature of the worm's spread, after a couple of hours, 1 million user pages would host the worm.

Samy had a million friend requests.

myspace

Samy is my Hero worm



Epilogue

- Samy was prosecuted under the Patriot Act.
- He pleaded guilty to a felony and was sentenced to three years of probation (without internet access), 90 days of community service, as well as paying MySpace an undisclosed amount of money in restitution.

What kind of attack was Samy?

A virus

A rootkit

A Cross Site Scripting exploit (XSS)

A Trojan Horse

Cross Site Scripting (XSS)

- An XSS vulnerability allows malicious users to attack other users of a service by injecting their own code in the service.
- Several types of these attacks exist. The main one are **persistant** (like in the case of Samy), or **reflected**, but more exist.
- XSS exploits often allow for the use of the permission of an authenticated user to perform operations without their knowledge. (This is sometimes called horizontal privilege escalation.)

Challenge 4.2 (Simple): Samy

- Go read the sources of Samy and/or the explanations of its author to answer the following questions:
- Myspace prevented users to add most html tags such as <script>. So where did Samy put his code?
- Myspace prevented users from using essential javascript keywords such as innerHTML, how did Samy bypass this limitation?
- Write a one-sentence explanation for each of these questions.

Vertical Privilege Escalation

- Occurs when an attacker gains access to higher-level permissions or privileges than they are authorized for, such as a regular user obtaining admin rights.
- Common causes include:
  - Misconfigured access control (e.g., APIs or admin panels without proper restrictions).
  - Exploiting software vulnerabilities (e.g., buffer overflow or insecure session management).
  - Weak authentication mechanisms, such as predictable passwords for admin accounts.

Key concepts

- Linux permissions
- OAuth
- XSS
- Vertical vs Horizontal Privilege escalation

Saltzer and Schroeder’s Security Design Principles

- In 1975, two security researchers, Saltzer and Schroeder, published an important paper that proposed 10 general design principles to build secure computer systems.
- See R. E. Smith, "A Contemporary Look at Saltzer and Schroeder's 1975 Design Principles," in IEEE Security & Privacy

Economy of mechanism

A simpler design is easier to test and validate.

Fail-safe defaults

The default values should lead to a secure behavior.
(Deny by default)

Complete mediation

Access rights are completely validated every time an access occurs.

Open design

"The enemy knows the system." No security through obscurity.

Separation of privilege

Set multiple independent conditions and associated checks before authorizing an operation.

Pick one of these principles and discuss one aspect of the previous lectures (or this one) where it applies (or is violated).

Fail-safe defaults

Open design

Separation of privilege

Economy of mechanism

Least privilege

Every program and user should operate while invoking as few privileges as possible.

Least common mechanism

Isolate components when possible to avoid unintended communication paths.

Psychological acceptability

Users should not have to work against (or around) the sytem.

Work factor

Make attacks expensive.

Compromise recording

The system should keep attack records even if the attacks aren’t blocked.

Pick one of these principles and discuss one aspect of the previous lectures (or this one) where it applies (or is violated).

Least privilege

Least common mechanism

Psychological acceptability

Work factor

Key concepts

- All ten design principles

Questions?

Voting cards



- Take one at the beginning of the lecture.
- Please, **do not crumple them nor write on them**.
- Please, **return them** at the end of the lecture.